

(green button with + icon) in your Thonny IDE and enter this Python code there:

```
# onboard LED blink code
from machine import Pin
from time import sleep

pled = Pin(25, Pin.OUT)
pled.on()
sleep(1)
pled.off()
```

Now just hit the *Run* button on the Thonny IDE and you should see the LED on your Pico board light up for a second. Congratulations! You have started your IoT journey with the RPI Pico board. The logic of the MicroPython code shown above is simple: we set the GPIO pin 25 as an output pin, make the voltage high to ‘on’ the onboard LED and ‘off’ it after a delay to witness the LED glowing up. You can replace the pin number 25 provided in the pin constructor with the string ‘LED’ too to achieve the same result. You can also use the functions *value(1)* and *value(0)* instead of *on()* and *off()*, respectively, to make the voltage on the onboard LED pin high and low.

Let’s make the project more interesting by introducing a loop that will make the onboard LED blink continuously. The code for this is:

```
# onboard LED flashing code
from machine import Pin
from time import sleep

pled = Pin("LED", Pin.OUT)
while True:
    pled.toggle()
    sleep(1)
```

Now click on the *Run* button and the onboard LED should flash continuously until you hit the *Stop* button in the IDE. Feel free to experiment with decimal shorter values in *sleep()* as that gives a nice flashing effect. The logic of the code is self-explanatory: we’re running in an infinite loop with the onboard LED toggling ‘on’ to ‘off’ and vice versa on each run with a delay.

Let’s now work with an onboard hardware timer available on the Pico board to flash the onboard LED. Stop the existing code running in the IDE and run the code shown below to flash the LED through the onboard timer:

```
# h/w timer based LED flash
from machine import Pin, Timer
```

```
led = Pin("LED", Pin.OUT)
tim = Timer()

def tick(timer):
    global led
    led.toggle()

tim.init(freq=10, mode=Timer.PERIODIC, callback=tick)
```

The logic of the code is simple: a hardware timer on the Pico microcontroller is periodically invoking the onboard LED toggling. You can use a *period=<time in milliseconds>* in *init()* instead of *freq* and the timer. The timer runs in the background to invoke *tick(...)* at periodic intervals only. This code is non-blocking, and you’ll see a MicroPython prompt in the Thonny IDE to enter more code.

Wouldn’t it be nice to see the onboard LED blink in a fading up/down manner, for a change? The Pico board has pulse width modulation functionality for that. The code given below, taken from official Pico-MicroPython examples, makes the onboard LED blink in a fading mode:

```
# PWM based LED fading

import time
from machine import Pin, PWM

# Construct PWM object, with LED on Pin(25).
pwm = PWM(Pin(25))

# Set the PWM frequency.
pwm.freq(1000)

# Fade the LED in and out a few times.
duty = 0
direction = 1
for _ in range(8 * 256):
    duty += direction
    if duty > 255:
        duty = 255
        direction = -1
    elif duty < 0:
        duty = 0
        direction = 1
    pwm.duty_u16(duty * duty)
    time.sleep(0.001)
```

The Pico board has analogue-to-digital converters to measure analogue signals and convert them to digital numbers to hand over to the microcontroller for processing in multiple ways. There is also a temperature sensor on the microcontroller chip connected to an ADC channel to report its temperature. This sensor gives a rough idea about